

MACHINE LEARNING BOOTCAMP · DAY 4 OF 5



Deep Learning & Computer Vision

Neural networks, CNNs, and teaching computers to see

Soyinka Enoch Oluwatimilehin · 01/10/2026

Quick recap of Day 3

1

Cross-validation gives fair scores, not lucky ones

2

Try simple first: logistic regression won on breast cancer

3

No labels? K-Means, PCA, anomaly detection

Warm-up: why would a random forest struggle to spot a cat in a photo?

Today's plan — 3 hours

Time	Block	What we'll do
0:00 – 0:10	Welcome	Day 3 recap, today's plan
0:10 – 0:55	Neural networks	Neurons, layers, learning + Notebook 1
0:55 – 1:25	Training well	Overfitting, dropout + Notebook 1
1:25 – 1:35	Break	Stretch, questions
1:35 – 2:20	CNNs	Convolutions, transfer learning + Notebook 2
2:20 – 2:52	Computer vision	YOLO and OpenCV + Notebook 3
2:52 – 3:00	Wrap-up	Recap, questions

PART 1

Neural networks

From one artificial neuron to deep learning.

Why deep learning?

Classic models need clean columns. Images, sound and text are raw and messy.

Images

Thousands of pixels. The object can be anywhere, any size.

Sound

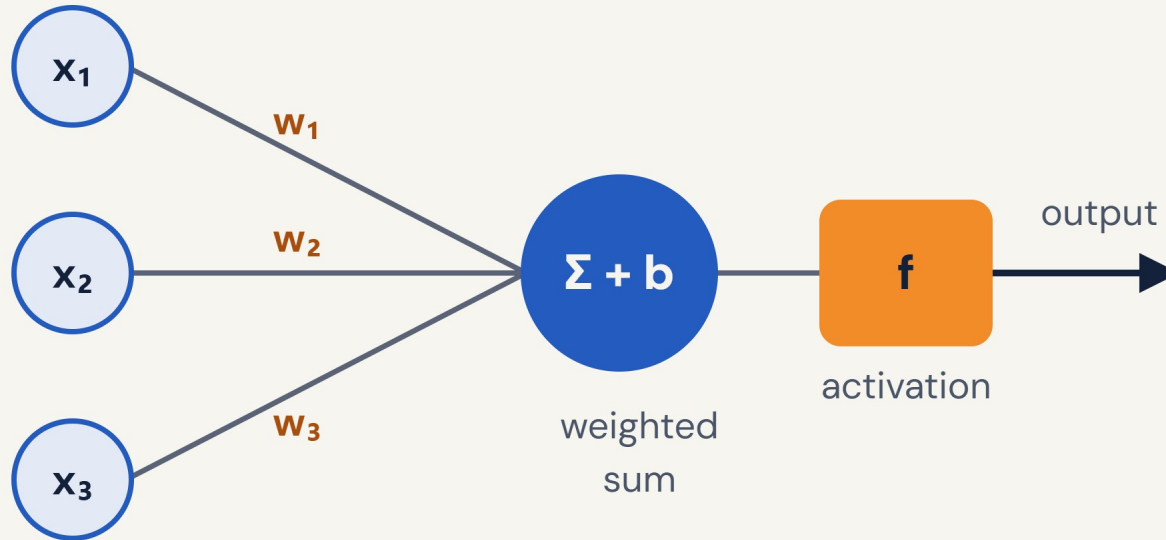
Thousands of samples every second.

Text

Meaning depends on word order and context.

Deep learning learns its own features from raw data.

An artificial neuron



Should I go out tonight?

Sunny: $1 \times \text{weight } 2$

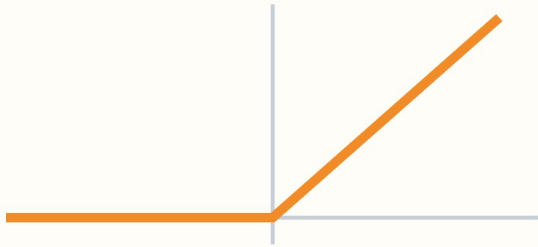
Have money: $1 \times \text{weight } 3$

Exam tomorrow: $1 \times \text{weight } -4$

Bias: -0.5

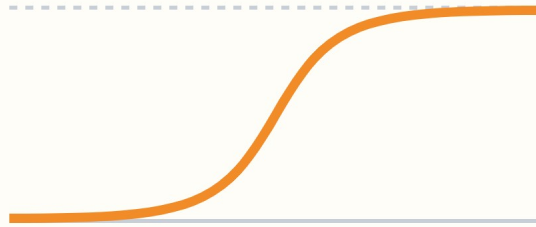
Sum = **0.5** $\rightarrow$ above 0 $\rightarrow$ **go**

Activation functions



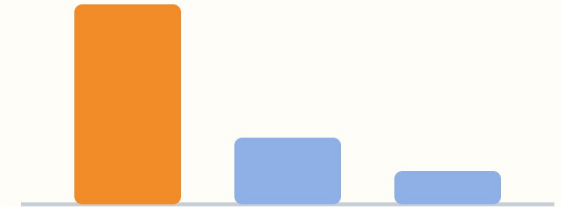
ReLU

Negative $\rightarrow$ 0, positive stays.
The default for hidden layers.



Sigmoid

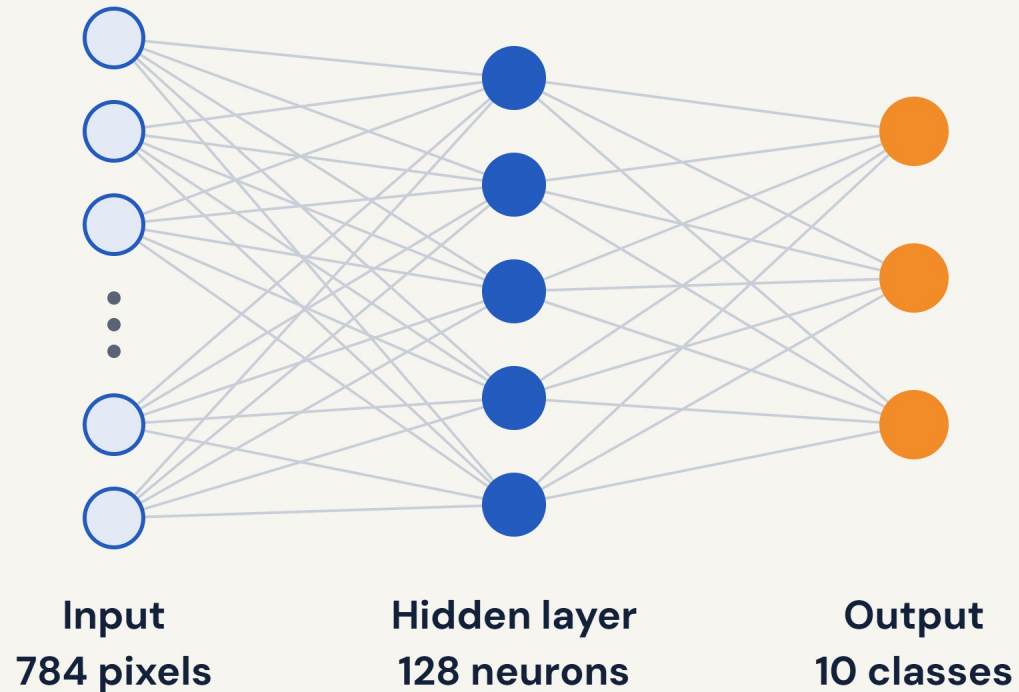
Squashes to 0–1. For
yes/no outputs.



Softmax

Turns scores into probabilities
that add to 1. For many classes.

Stacking neurons: a network



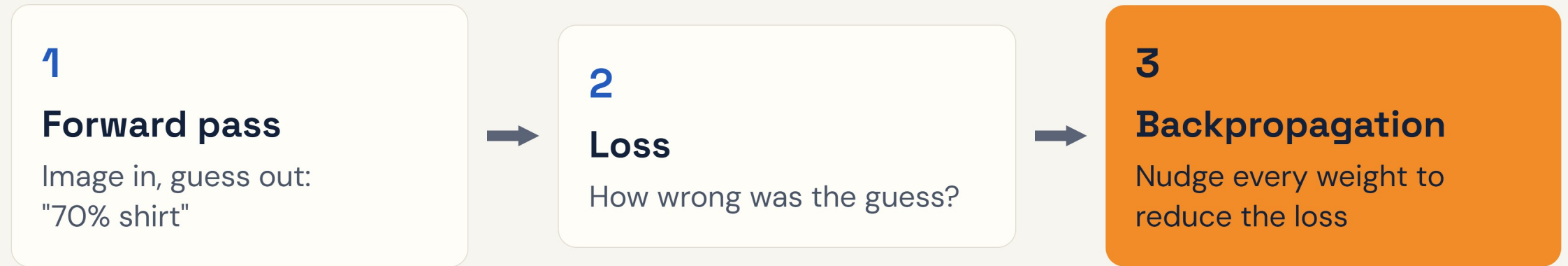
Every neuron connects to every neuron in the next layer (**Dense**)

Deep = many hidden layers

Each layer learns **more complex features**

Our first model: **101,770** weights to learn

How a network learns



Gradient descent: walking downhill in fog — feel the slope, take a small step down, repeat.

Training vocabulary

Word	Meaning
Epoch	One pass through all the training images
Batch	A small group of images per update (e.g. 32)
Learning rate	How big each downhill step is
Loss	How wrong the model is — lower is better
Validation set	Data held back to watch for overfitting
Optimizer (Adam)	The algorithm that takes the steps

54,000 training images ÷ 32 per batch = **1,688 weight updates per epoch**

A neural network in Keras

```
model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=10, validation_split=0.1)
```

Flatten: $28 \times 28 \rightarrow 784$

Dense 128: hidden layer

Dense 10: one per class

Same recipe as scikit-learn: **build** → **compile** → **fit** → **evaluate**

</> HANDS-ON · 15 MINUTES

Notebook 1 · Parts 1–3

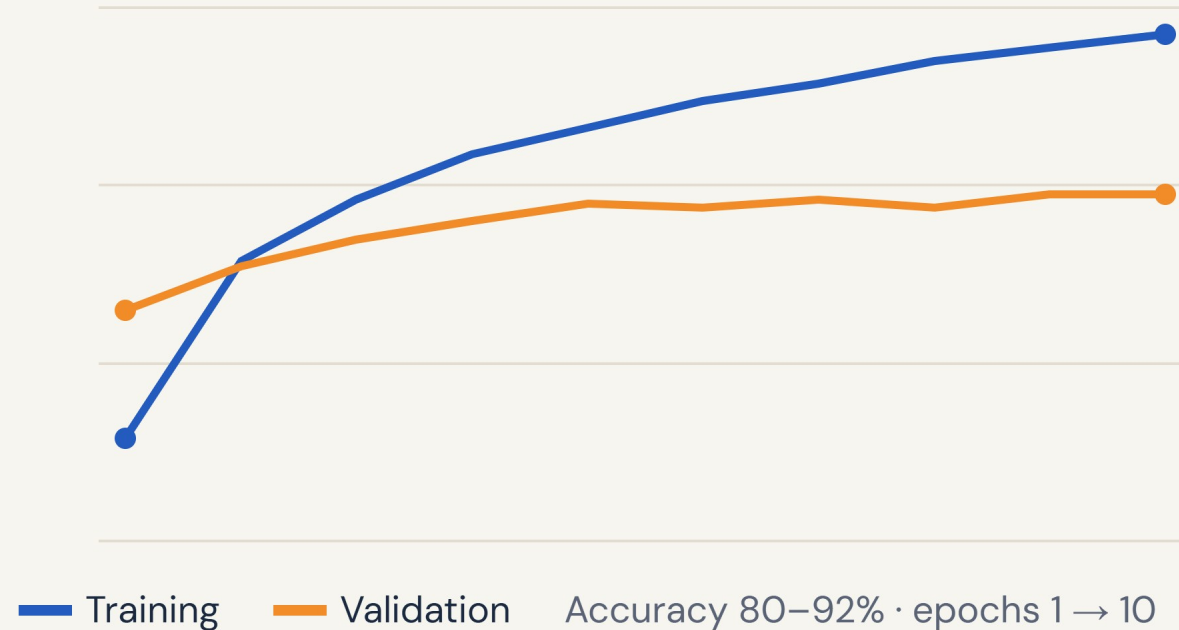
Day4_1_Neural_Networks.ipynb · GPU runtime

Part 1 · One neuron by hand

Part 2 · Meet Fashion-MNIST

Part 3 · Build, train and test your first network

Reading training curves



Both rising: still learning

Training rises, validation flat: starting to memorise

Test accuracy: **87.1%** after 10 epochs

Overfitting & dropout

A bigger network (2×512 neurons), trained for 30 epochs:

No dropout

Training: **96.6%** · Validation: **88.7%**

Validation loss got **worse**: 0.34 → 0.50

With dropout (30%)

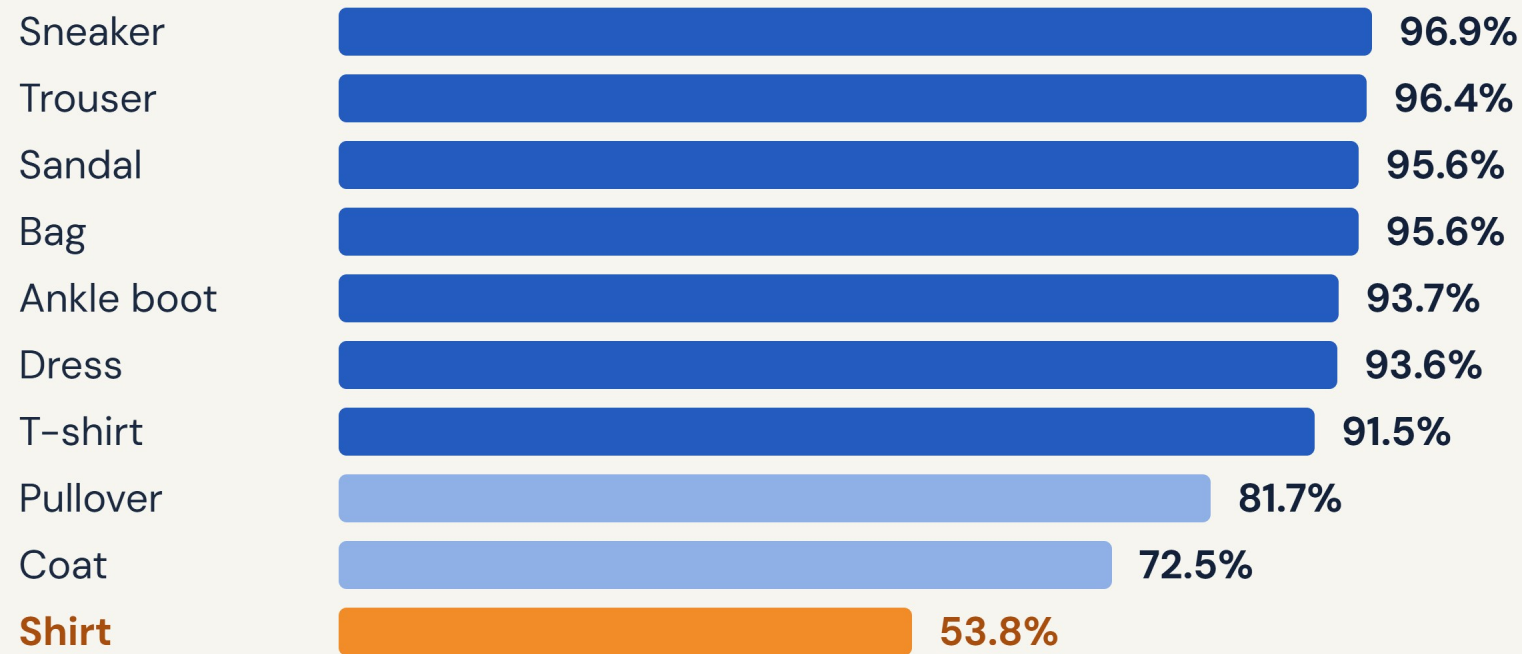
Training: **92.1%** · Validation: **89.6%**

Validation loss stayed **low**: 0.30 → 0.31



Dropout switches off random neurons during training — like a team practising with random players missing.

What does the network get wrong?



Shirts were called T-shirts 249 times out of 1,000 — at 28×28 pixels they look alike.

</> HANDS-ON · 15 MINUTES

Notebook 1 · Parts 4–5

Day4_1_Neural_Networks.ipynb

Part 4 · What does it get wrong? (confusion matrix)

Part 5 · Make it overfit — then fix it with dropout

PART 3

Convolutional neural networks

Networks that look at images the way we do — piece by piece.

To a computer, an image is numbers

		80	200	200	80		
	60	255	255	255	255	60	
		120	255	255	120		
		120	255	255	120		
		120	255	255	120		
		140	255	255	140		

Greyscale: one number per pixel, 0 = black, 255 = white

Fashion-MNIST: $28 \times 28 = 784$ numbers

Colour: 3 numbers per pixel — Red, Green, Blue

A phone photo: $4000 \times 3000 \times 3 = 36$ million numbers

Convolution: a sliding pattern detector

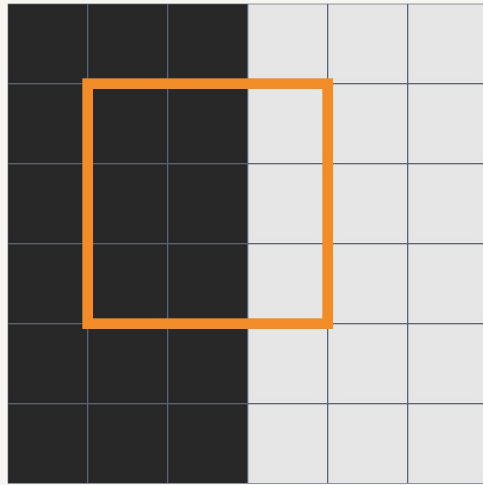


Image with an edge

×

-1	0	1
-1	0	1
-1	0	1

3 × 3 filter

Slide the filter over every spot. Multiply, add up.

Big answer where dark meets bright → an edge

The network **learns** its own filters

Pooling: shrink, keep the strongest

1	3	2	0
5	2	1	7
0	1	4	2
6	2	3	1

→

5	7
6	4

Max pooling 2×2 : keep the biggest number in each block

Image gets 4× smaller → faster

Small shifts don't matter

Our CNN, layer by layer



Feature extractor

Conv + pool, twice: edges
→ shapes

Classifier

Dense layers turn the shapes
into a decision

225,034 weights

Most of them (204,928) in the
first dense layer

```
Conv2D(32, 3) → MaxPool → Conv2D(64, 3) → MaxPool → Flatten → Dense(128) → Dropout(0.3) → Dense(10)
```

CNN vs dense network

Dense network

87.1%

CNN

91.0%

Shirt: 53.8% → **68.6%** · **Coat:** 72.5% → **86.5%** · **Pullover:** 81.7% → **86.3%**

Same data, 10 epochs each. The CNN makes **30% fewer mistakes** (1,290 → 900 of 10,000).

Transfer learning: don't start from zero



1 • Pre-train

Someone trains a big model on ImageNet: 1.2 million photos, 1,000 classes



2 • Reuse

Keep its learned filters — edges, textures, shapes



3 • Fine-tune

Retrain on your small dataset for a few minutes



Like hiring an experienced mechanic instead of training someone who has never seen a car.

Pre-trained vs from scratch

Imagenette: 10 classes of real photos (fish, dog, church, parachute ...) · same model · 2 epochs

Pre-trained



92.2%

From scratch



30.0%

Pre-trained knowledge is the shortcut.

</> HANDS-ON · 16 MINUTES

Notebook 2

Day4_2_CNN_Transfer_Learning.ipynb · GPU runtime

Part 1 · Images are numbers

Part 2 · Convolution with your own filter

Part 3 · Train a CNN on Fashion-MNIST

Part 4 · Transfer learning: pre-trained vs scratch

PART 4

Computer vision

Teaching machines to see: find, outline and track what is in an image.

Four big vision tasks



Classification

What is in the picture? → “bus”



Detection

What and **where**? → boxes + labels



Segmentation

Which exact **pixels** belong to each object?



Pose estimation

Where are the body **joints**? 17 key points

YOLO: You Only Look Once

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
results = model("bus.jpg")
results[0].show()
```

One pass through the network finds **every** object — fast enough for live video

Result on bus.jpg

bus · **0.94**

person · **0.89 · 0.88 · 0.86 · 0.62**

~110 ms per image on a
laptop CPU

2.6 M weights · 80 object types

OpenCV: the image toolbox



Read & resize

Load images, video and webcams; crop, resize, rotate



Filters

Blur, sharpen, convert to grey, find edges (Canny)



Draw

Boxes, text and lines on top of results

```
img = cv2.imread("bus.jpg") # note: BGR, not RGB!  
edges = cv2.Canny(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 100, 200)
```

Computer vision around us



Agriculture

Spot cassava & maize disease from a phone photo



Health

Read X-rays, find malaria parasites in blood smears



Traffic

Read number plates, count cars, catch red-light runners



Security

Face unlock, attendance systems, ID checks



Retail

Self-checkout, shelf stock, visual search



Robots & drones

Navigate, avoid obstacles, pick objects

Beyond images: sequences



RNN / LSTM

Read data in order,
remembering the past —
sensor signals, speech, prices



Transformers

Attention: look at every word
at once. Powers ChatGPT,
Claude, Gemini



Vision Transformers

The same idea for images —
cut a photo into patches

Tomorrow: generative AI builds on all of this.

</> HANDS-ON · 13 MINUTES

Notebook 3

Day4_3_Computer_Vision.ipynb

Part 1 · OpenCV basics: colours, edges, drawing

Part 2 · Detect objects with YOLO

Part 3 · Segmentation and pose

Part 4 · Try it on your own photo

Recap: what we covered today

- 1 A neuron: weighted sum + bias $\rightarrow$ activation
- 2 Training: loss, backpropagation, epochs — watch the curves
- 3 Dropout fights overfitting
- 4 CNNs: filters + pooling $\rightarrow$ 91% on Fashion-MNIST
- 5 Transfer learning and YOLO: reuse, don't rebuild

BEFORE DAY 5

- Finish the 3 Day 4 notebooks
- Add a third Conv layer — does accuracy improve?
- Run YOLO on 3 photos of your own

Tomorrow: ML on ESP32 & Raspberry Pi, biomedical signals, robotics, RL and GenAI

Questions?

Tomorrow: ML on devices, robotics, RL & GenAI.

Soyinka Enoch Oluwatimilehin · 08144801748